

# 2

## プログラミング概論I (オンデマンド授業)



担当: 中山 功一 (7号館3階308室)

メールアドレス: [knakayama@is.saga-u.ac.jp](mailto:knakayama@is.saga-u.ac.jp)

86

### オープニングテスト

前回, 習ったことを復習するために,  
オープニングテストを行います

問題ごとに動画を止めて, 答えをメモしましょう.

最後に, 答えを動画で表示しますので,  
自分が理解しているか, 確認しましょう.

### オープニング テスト

問1~4: 各問で, 画面に表示される文字は?

```
#include <iostream>
using namespace std;
int a = 3, b = 2, c = 1;
cout << b; //問1
b = a;
cout << b; //問2
b = c;
cout << b; //問3
cout << 'b'; //問4
cout << endl;
```

<選択肢>

- ① 1
- ② 2
- ③ 3
- ④ a
- ⑤ b
- ⑥ c
- ⑦ その他

### オープニング テスト

次のプログラムについて答えなさい  
問5~7: 画面に表示される文字は?

```
#include <iostream>
using namespace std;
int a = 1, b = 2, Hello = 3;
cin >> b;
cout << 'b'; //問5
cout << a; //問6
cout << Hello; //問7
cout << endl;
```

<選択肢>

- ① 1
- ② 2
- ③ 3
- ④ a
- ⑤ b
- ⑥ c
- ⑦ その他

### 答え合わせ

全ての答えをメモしましたか?

## 解答

- 【1】 ② bは, 初期値のまま
- 【2】 ③ bには, aの値:3が代入済み
- 【3】 ① bには, cの値:1が代入済み
- 【4】 ⑤ ' 'があるので文字bが表示される
- 【5】 ⑤ ' 'があるので文字bが表示される
- 【6】 ① aは, 初期値のまま
- 【7】 ③ 変数Helloの値は初期値3のまま

92

## 文字型と文字列クラス

- **char型**
  - 1文字(半角文字)を格納する
  - データサイズ:1バイト
  - 注: 数字(文字)は数値ではない!
- **stringクラス**
  - 文字列(全角文字も含む)を格納する
  - #include <string> が必要
  - データサイズ: 指定された文字列を格納するのにちょうど良いサイズを自動設定

93

## 文字型(char型)

表示      10進数      1バイト(8bit)文字  
ア      177      10110001  
変換      変換

char 1バイト(8bit)

|    | 00 | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | A0 | B0 | C0 | D0 | E0 | F0 |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 00 | NL | DE | SP | 0  | @  | P  | *  | p  |    |    |    |    |    | —  | タ  | ミ  |
| 01 | SH | D1 | !  | 1  | A  | Q  | a  | q  |    |    |    |    |    | .  | ア  | チ  |
| 02 | SX | D2 | "  | 2  | B  | R  | b  | r  |    |    |    |    |    | †  | イ  | ツ  |
| 03 | EX | D3 | #  | 3  | C  | S  | c  | s  |    |    |    |    |    | ‡  | ウ  | テ  |
| 04 | ET | D4 | \$ | 4  | D  | T  | d  | t  |    |    |    |    |    | .  | エ  | ト  |
| 05 | ES | WK | R  | 5  | E  | U  | e  | u  |    |    |    |    |    | .  | オ  | ヲ  |
| 06 | AK | SN | 6  | 6  | F  | V  | f  | v  |    |    |    |    |    |    | ヲ  | カ  |
| 07 | BL | EB | 7  | 7  | G  | W  | g  | w  |    |    |    |    |    |    | ア  | キ  |
| 08 | SS | CN | 8  | 8  | H  | X  | h  | x  |    |    |    |    |    |    | イ  | ク  |
| 09 | HT | EM | 9  | 9  | I  | Y  | i  | y  |    |    |    |    |    |    | ウ  | ケ  |
| 0A | LF | SB | K  | :  | J  | Z  | j  | z  |    |    |    |    |    |    | エ  | コ  |
| 0B | HV | EC | +  | :  | K  | [  | k  | [  |    |    |    |    |    |    | オ  | サ  |
| 0C | CL | →  | .  | <  | L  | ¥  | l  | l  |    |    |    |    |    |    | ヤ  | シ  |
| 0D | CR | ←  | —  | =  | M  | ]  | m  | ]  |    |    |    |    |    |    | ス  | ヘ  |
| 0E | SO | ↑  | .  | >  | N  | ^  | n  | ^  |    |    |    |    |    |    | セ  | ホ  |
| 0F | SI | ↓  | /  | ?  | O  | _  | o  | _  |    |    |    |    |    |    | ン  | マ  |

## 論理型(bool型)

- 論理型の値を表す組み込み定数
  - true : 真 (1で表す)
  - false : 偽 (0で表す)
- データ型の名前は bool
- サイズは1バイト(処理系に依存)

95

96

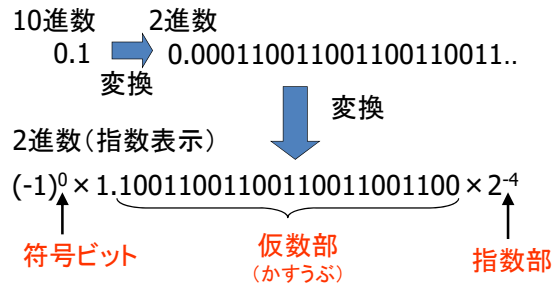
## 実数型(小数点を含む数値)

- 小数点を含む値を格納するためのデータ型
  - **double**: 倍精度浮動小数点(8バイト:16ケタ)
  - 授業で最も良く使う
  - **float**: 単精度浮動小数点(4バイト:7ケタ)
  - **long double**: 四倍精度浮動小数点(16バイト)

注: 有効数字と範囲の違いに注意

97

## 実数型の表現



98

## 実数型

- 小数值や実数値を表現する
  - 浮動小数点実数
- 実数のためのデータ型

2進数(指数表示)

$(-1)^0 \times 1.10011001100110011001100 \times 2^{-4}$

符号ビット (1ビット)      仮数部 (23ビット)      指数部 (8ビット)

float(4バイト)      23ビット      8ビット

double(8バイト)      52ビット      11ビット

通常はこれ

注: バイト数は処理系に依存

## 2進数の位

<10進数の場合>

7 7 7.7 7 7

↑ ↑ ↑ ↑ ↑

$10^2$   $10^1$   $10^0$   $10^{-1}$   $10^{-2}$   $10^{-3}$

の の の の の の

位 位 位 位 位 位

<2進数の場合>

1 1 1.1 1 1

↑ ↑ ↑ ↑ ↑

$2^2$   $2^1$   $2^0$   $2^{-1}$   $2^{-2}$   $2^{-3}$

の の の の の の

位 位 位 位 位 位

## 参考: 16進数

| 10進数 | 1~9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|------|-----|----|----|----|----|----|----|----|
| 16進数 | 1~9 | A  | B  | C  | D  | E  | F  | 10 |

<10進数の場合>

7 7 7.7 7 7

↑ ↑ ↑ ↑ ↑

$10^2$   $10^1$   $10^0$   $10^{-1}$   $10^{-2}$   $10^{-3}$

の の の の の の

位 位 位 位 位 位

<16進数の場合>

0x F F F.F F F

↑ ↑ ↑ ↑ ↑

16進数の印

$16^2$   $16^1$   $16^0$   $16^{-1}$   $16^{-2}$   $16^{-3}$

の の の の の の

位 位 位 位 位 位

## ボーナス問題 その1

- 4分の1(10進数で0.25)を2進数で表現すると?

正解は...

0.01 ←  $2^{-2}$ の位(1/4)

↑

$2^{-1}$ の位(1/2)

102

## ボーナス問題 その2

- 5分の1 (10進数で0.2)を  
2進数で表現すると？

正解は...

0.0011001100...



$1/8 + 1/16 + 1/128 + 1/256 + \dots$

103

104

## 実数型の誤差

## 実数型の誤差

- 以下のプログラムを実行すると？

```
double x, y;
x = 1.0/10.0;
x = x + x + x;
y = 0.3;
if(x == y) cout << "O" << endl;
else cout << "X" << endl;
```

C:\¥WINDOWS¥system32¥cmd.exe  
X  
続行するには何かキーを押してください .

## 実数型の誤差

- 以下のプログラムを実行すると？

```
double x, y;
x = 1.0/10.0;
x = x + x + x;
y = 0.3;
cout << "x=" << x << ", y=" << y << endl;
if(x == y) cout << "O" << endl;
else cout << "X" << endl;
```

xとyの表示:確認用

C:\¥WINDOWS¥sy  
x=0.3, y=0.3  
X  
続行するには何かキー

## 実数型の誤差

- 以下のプログラムを実行すると？

```
double sum = 0.0;
for(int i=0; i<10; i++){
    cout << sum << endl;
    sum += 0.1;
}
```

108

```

C:\¥WINDOWS¥system32¥cmd.exe
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
続行するには何かキーを押してください . . .

```

109

## 実数型の誤差

- 以下のプログラムを実行すると？

```

double sum = 0.0;
for(int i=0 ; i<10 ; i++){
    cout.precision(17);
    cout << sum << endl;
    sum += 0.1;
}

```

出力精度を  
17桁に設定

110

```

C:\¥WINDOWS¥system32¥cmd.exe
0
0.100000000000000001
0.200000000000000001
0.300000000000000004
0.400000000000000002
0.5
0.59999999999999998
0.69999999999999996
0.79999999999999993
0.89999999999999991
続行するには何かキーを押してください . . .

```

111

## 実数の処理

- 値を有限桁の二進小数で表現する  
→ 誤差が出る！！
- 無限小数は近似値で表す
  - 10進表現では有限小数でも、  
2進表現では無限小数になるかも！

112

## 誤差の分類

- ① 丸め誤差とその累積
- ② オーバーフロー
- ③ アンダーフロー
- ④ 桁落ち

113

### ① 丸め誤差とその累積

10進数で有限小数でも、2進数では無限小数かも  
丸め誤差＝| 厳密値－近似値 |

| 10進数                       | 2進数                | 丸め誤差            |
|----------------------------|--------------------|-----------------|
| 0.5                        | 0.1                | なし              |
| 0.1                        | 0.0001100110011... | あり              |
| 0.1+...+0.1<br>(1000回)=100 | 100に近い実数値          | あり<br>(累積1000倍) |

<対策>

- できるだけ整数演算にする
- 反復加算を避ける

114

## ② オーバーフロー

絶対値が表現可能な値より大きくなる



不正な値が代入される！！

(整数演算の場合も発生する)

- 例
- int型(16ビット)の変数に100000を代入
  - float型(16ビット)の変数に  
 $1.1111 \cdot 1 \times 2^{127}$ より大きな値を代入

<対策>

- ・適切なデータ型を用いる
- ・計算順序を工夫する

115

## ③ アンダーフロー

絶対値が表現可能な値よりも小さくなる



近似値:0が代入される！！

- 例
- float型(16ビット)の変数に  
 $1.0000 \cdot 0 \times 2^{-16}$ より小さな値を代入する

<対策>

- ・適切なデータ型を用いる
- ・計算順序を工夫する

116

## ④ 桁落ち

似た値同士を減算し、  
有効桁数が極端に小さくなる

例

$$\begin{array}{r} 1.11111111111111 \times 2^{16} \\ - 1.11111111111110 \times 2^{16} \\ \hline 0.00000000000001 \times 2^{16} \\ = 1.00000000000000 \times 2^3 \end{array}$$



有効桁は最初の1桁のみ

<対策>

- ・計算順序を工夫する

117

## 型変換

## 型変換 (type casting)

- 型変換とは？
  - 異なるデータ型に変換すること
- 型変換はどのような時に起こるか？
  - 式(右辺)と変数(左辺)でデータ型が異なる代入
  - 異なるデータ型の変数/値を使った式の計算
- 型変換はどのように実行されるか？
  - 暗黙の型変換(コンパイラが自動的に型変換)
  - 明示的な型変換(プログラマが指定して型変換)

119

## 式と変数のデータ型が異なる代入文

式のデータ型を変数の  
データ型に合わせる

原則: 型を一致させる  
(不一致はバグの原因)

例1: 暗黙の型変換

```
int i;  
i = 2.5;
```

- ① double型の2.5を整数化
- ② 2.5→2になる
- ③ 2をiに代入

例2: 明示的な型変換  
(キャスト、キャストイング)

```
float x;  
x = (float)2.5;
```

- ① double型の2.5を明示的にfloat型に変換
- ② float型の値をxに代入

120

## 異なるデータ型の値を使った式の計算

### <暗黙の型変換規則>

- 小数はdouble型
- データ型が複数なら、最も表現範囲の広い型に自動変換

short < int < long

< float < double

使わないこと

例1: 1/3

– 整数の除算: 結果は0

例2: (double)1/(double)3

– 明示的な型変換

– 計算結果は0.333...

例3: 1/3.0

– 暗黙の型変換で

double型に変換

– 計算結果は0.333...

## 変更しない箱1: 定数

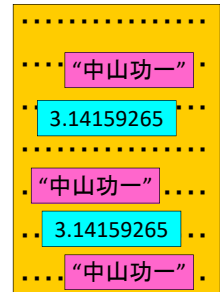
### • 数学の例:

- 円周率を $\pi$ と置く
- ネイピア数を $e$ と置く

### • プログラムの例:

- PI 3.14159265
- NAPIER 2.71828182
- MY\_NAME 中山功一

代入不可, 参照のみOK



122

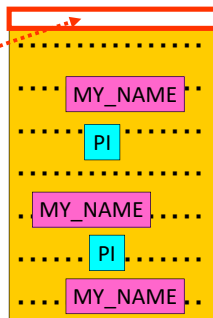
## 変更しない箱1: 定数

- 以下のような値は、プログラム中に直接書くと面倒
  - 何度も出てくる値
  - 後で、頻繁に変更する値
- 別名を、最初に「宣言」しておくことができる(プログラム実行中に変更できない)

### 宣言

MY\_NAME は “中山功一”

PI は 3.14159265



123

## 変更しない箱1: 定数

### 構文:

#define 定数名 値

#defineの最後には「;」をつけない

代入ではないので「=」をつけない

```
#define N 10 // 教員が指定した値
#define PI 3.1416 // πの値
#define NUM_STUDENT 6000 // 学生総数
#define ZIP_CODE 8408502 // 郵便番号
```

### <利点>

- 値の意味が分かりやすい
- 値の一括変更が可能

124

## 変更しない箱2: const修飾子

### const変数定義の方法

const 型名 変数名;

```
const int n = 10; // 教員が指定した値
const double pi = 3.1416; // πの値
const int num_student = 6000; // 学生総数
const int zip_code = 8408502; // 郵便番号
```

### <利点>

- 変数のまま、値を固定できる

125

## 定数と変数

### <定数> #define 定数名 値

- コンパイル時に値が決定する
- 代入文では値を変更できない

### <const変数> const 型名 変数名;

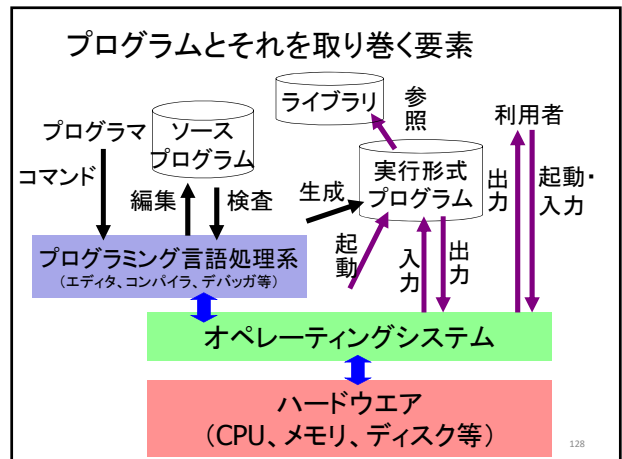
- 定義時に値が決定する
- 代入文では値を変更できない

### <変数> 型名 変数名;

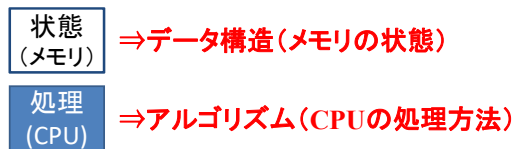
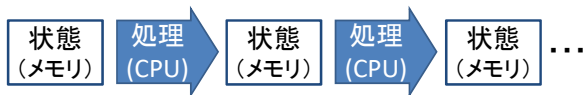
- 実行時点で値が決定する
- 代入文で値を変更できる

126

## プログラムとは？



## プログラムとは



これらをPCに伝える言葉がプログラム

## 状態遷移を示す「トレース表」

- 全ての **変数** を 1行目に記述
- **変数** の値の変化を処理ごとに各行に記述

変数: データを格納する場所

| Step    | a | b | c | d |
|---------|---|---|---|---|
| ① 初期値   | 1 | 2 | 3 | 4 |
| ① a=d   | 4 | 2 | 3 | 4 |
| ② d=a+b | 4 | 2 | 3 | 6 |
| ③ cout  | 4 | 2 | 3 | 6 |
| ④ b++   | 4 | 3 | 3 | 6 |
| ⑤ c+=a  | 4 | 3 | 7 | 6 |

130

## 例題1

```
int a = 3, b = 2, c = 1;
b = a; //step1
b = c; //step2
```

| ステップ | a | b | c |
|------|---|---|---|
| 0    | 3 | 2 | 1 |
| 1    | 3 | 3 | 1 |
| 2    | 3 | 1 | 1 |

131

## 例題2

```
int a = 1, b = 2, c = 3;
a = c; //step1
c = 5; //step2
b = c; //step3
```

| ステップ | a | b | c |
|------|---|---|---|
| 0    | 1 | 2 | 3 |
| 1    | 3 | 2 | 3 |
| 2    | 3 | 2 | 5 |
| 3    | 3 | 5 | 5 |

132



## 演算

## 四則演算子

| 演算子        | 名前    | 意味                                         | 計算順序 |
|------------|-------|--------------------------------------------|------|
| +          | 加算演算子 | 足し算の結果:和を返す                                | 後    |
| -          | 減算演算子 | 引き算の結果:差を返す                                | 後    |
| *          | 乗算演算子 | 掛け算の結果:積を返す                                | 先    |
| /          | 除算演算子 | 割り算の結果:商を返す<br>注:整数型の割り算では、<br>小数点以下“切り捨て” | 先    |
| %<br>(mod) | 剰余演算子 | 割り算の余りを返す                                  | 先    |

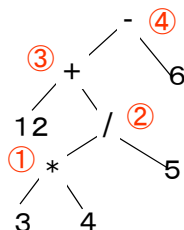
優先順位に迷ったら「( ) = 最優先」をつけること<sup>134</sup>

## 四則演算子の優先順位

例  $12+3*4/5-6$

括弧で順序を変更できる

例  $(12+3)*4/(5-6)$



赤字: 演算の実行順序

135

## 四則演算子の結合規則

同じ優先順位の四則演算子は左から順に計算する

| プログラムの表記    | プログラムの意味                 |
|-------------|--------------------------|
| $1+2+3+4+5$ | $((((1+2)+3)+4)+5) = 15$ |
| $1+2-3+4-5$ | $((((1+2)-3)+4)-5) = -1$ |
| $1-(2-3)-4$ | $1-((-1)-4) = -2$        |
| $16/4/2$    | $(16/4)/2 = 4/2 = 2$     |
| $16/(4/2)$  | $16/2 = 8$               |
| $16/4\%2$   | $(16/4)\%2 = 4\%2 = 0$   |

136

## 代入演算子

| 代入演算子           | 意味                            |
|-----------------|-------------------------------|
| $a = \text{値}$  | $a$ に値を代入する                   |
| $a = \text{式}$  | $a$ に式の値を代入する                 |
| $a++$           | $a$ を1増やす( $a=a+1$ )          |
| $a--$           | $a$ を1減らす( $a=a-1$ )          |
| $a += \text{式}$ | $a$ を式の値増やす( $a=a+\text{式}$ ) |
| $a -= \text{式}$ | $a$ を式の値減らす( $a=a-\text{式}$ ) |
| $a *= \text{式}$ | $a$ を式の値倍する( $a=a*\text{式}$ ) |
| $a /= \text{式}$ | $a$ を式の値で割る( $a=a/\text{式}$ ) |

137

## 優先順位の一覧

| 優先度  | 演算子                      | 計算順序  |
|------|--------------------------|-------|
| 1位   | ( )                      | 左から→  |
| 2位   | ++, --(インクリメント, デクリメント)  | ← 右から |
| 3位   | *, /, %(掛け算, 割り算, 剰余算)   | 左から→  |
| 4位   | +, - (足し算, 引き算)          | 左から→  |
| (5位) | <, <=, >, >=(比較演算子:不等号)  | 左から→  |
| (6位) | ==, != (比較演算子:等号)        | 左から→  |
| (7位) | &&(論理積)                  | 左から→  |
| (8位) | (論理和)                    | 左から→  |
| 9位   | =, +=, -=, *=, /=(代入演算子) | ← 右から |

優先順位に迷ったら「( ) = 最優先」をつけること<sup>138</sup>

## 注意: 優先順位の補足

インクリメント／デクリメント演算子や、  
代入演算子は、単独で使用するこ  
(複数の上記演算子を同時に使用しないこと)

悪い例: `a = b++; a = b += c;`

↑

例外的に++ (優先順位2位) が後、  
= (優先順位9位) が先に処理されます

139

## 代入演算子の結合規則

同じ優先順位の“代入”演算子は“右”から演算  
例 `int a = 1, b = 2, c = 3;`

| プログラムの表記                      | プログラムの挙動                                                                                      |
|-------------------------------|-----------------------------------------------------------------------------------------------|
| <code>a = b += c = 4;</code>  | (1) <code>c=4;</code><br>(2) <code>b += 4; //(b → 6)</code><br>(3) <code>a = 6;</code>        |
| <code>a = b + (c = 4);</code> | (1) <code>c に 4 を代入</code><br>(2) <code>a = b + c;</code><br>(3) <code>a に (2 + 4) を代入</code> |

このような分かりにくいコードを書かないこと 140

## 代入演算子の結合規則 (計算の順番)は右から左

- なぜ「右から左」になっているか?
- 結合規則の適用事例

`i = j = k = 0;`

上記の文は以下のように解釈される

`i=(j=(k=0));`

→ 一文でi, j, kの全てに値を設定できる

それ以外の結合規則は左から右

141

## 例題

箱a～箱dに、以下の数字が入っています  
次の命令文を順番に実行してください

① `a = b;` ② `b = c = d` ③ `c = b = a;`

| 箱 ( a ) | 箱 ( b ) | 箱 ( c ) | 箱 ( d ) |
|---------|---------|---------|---------|
| 2       | 2       | 4       | 4       |

142

## 例題

`int a=1, b=2, c=3, d=4;`  
`a = b; //step1`  
`b = c = d; //step2`  
`c = b = a; //step3`

| ステップ | a | b | c | d |
|------|---|---|---|---|
| ①    | 1 | 2 | 3 | 4 |
| ①    | 2 | 2 | 3 | 4 |
| ②-1  | 2 | 2 | 4 | 4 |
| ②-2  | 2 | 4 | 4 | 4 |
| ③-1  | 2 | 2 | 4 | 4 |
| ③-2  | 2 | 2 | 2 | 4 |

143

## 演算子:オペレータ, 被演算子:オペランド

### 単項演算子:

1つのオペランドを演算するオペレーター

例: ++ (1つのオペランドをインクリメントする)

### 二項演算子:

2つのオペランドを演算するオペレーター

例: + (2つのオペランドを加算する)

= (左のオペランドに右のオペランドを代入)

`a = b + c;` (1) + : b と c を加算する

(2) = : a に (1) の結果を代入する

144

## 条件分岐

### if文(条件分岐)

- 条件が成立／不成立で場合分け
- // 条件〇〇が成り立つならば、処理Aを実行

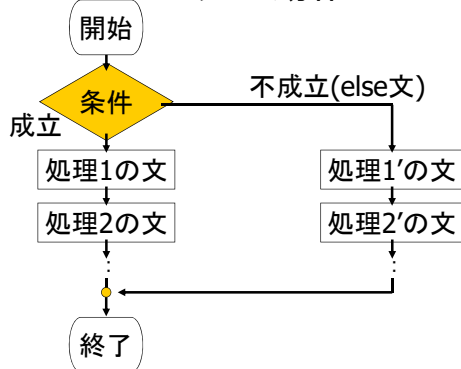
```
if(〇〇) {
    // 処理A
}
```

文が1つなら{, }は省略可  
(ただし省略しない方がよい)

```
// そうでなければ、処理Bを実行
else{
    // 処理B
}
```

処理がなければ省略可能

### if文の動作



### 比較を行なう演算子(関係演算子)

| 関係演算子      | 意味        | 優先順位 |
|------------|-----------|------|
| $a < b$    | aはbよりも小さい | 高    |
| $a > b$    | aはbよりも大きい | 高    |
| $a \leq b$ | aはb以下     | 高    |
| $a \geq b$ | aはb以上     | 高    |
| $a == b$   | aはbと等しい   | 低    |
| $a != b$   | aはbと等しくない | 低    |

- 関係演算子は四則演算子よりも優先順位は低い
- 関係演算子の結合規則は左から右(めったに使わない)

### 関係演算子(比較演算子)の処理

true 1, false 0

a=1の場合

```
if(a == 1) { ... }
if(true)   { ... }
if(1)      { ... }
```

a!=1の場合

```
if(a == 1) { ... }
if(false)  { ... }
if(0)      { ... }
```

```
if(a != 1) { ... }
if(false)  { ... }
if(0)      { ... }
```

```
if(a != 1) { ... }
if(true)   { ... }
if(1)      { ... }
```

### a = 1, b = 0の場合の処理の例

$((a == 1) == (b == 1))$

```

↓
((true) == (false))
↓
((1) == (0))
↓
(false)
↓
(0)
  
```

$((a != 1) != (b == 1))$

```

↓
((false) != (false))
↓
((0) != (0))
↓
(false)
↓
(0)
  
```

## 論理演算子

複数の条件を組み合わせて条件式を記述する

| 論理演算子                       | 意味                 | 説明                          | 優先順位 |
|-----------------------------|--------------------|-----------------------------|------|
| <code>a &amp;&amp; b</code> | <code>aかつb</code>  | aとbがともに真(少なくとも一方が偽ならば偽)     | 2    |
| <code>a    b</code>         | <code>aまたはb</code> | aとbの少なくとも一方が真(aとbがともに偽ならば偽) | 3    |
| <code>!a</code>             | <code>aでない</code>  | aが真ならば偽(aが偽ならば真)            | 1    |

## 論理型の演算

| X | Y | <code>!X</code> | <code>X &amp;&amp; Y</code> | <code>X    Y</code> |
|---|---|-----------------|-----------------------------|---------------------|
| 0 | 0 | 1               | 0                           | 0                   |
| 0 | 1 | 1               | 0                           | 1                   |
| 1 | 0 | 0               | 0                           | 1                   |
| 1 | 1 | 0               | 1                           | 1                   |

0 ... false  
1 ... true

& や | という演算子もあるが、これらはビット毎の論理演算子なので混同しない

## 論理演算子の使用例

// iが10以上20以下ならば以下の処理を実行する  
if ( (10<= i) && (i<=20) )

// iが10未満か、または20より大きいならば..  
if ( (i<10) || (20<i) )

// aとbがともに0ならば..  
if ( (a==0) && (b==0) )

// a, b, cがともに正ならば..  
if ( (a>0) && (b>0) && (c>0) )

## 論理演算子の使用例

// カードが赤の偶数ならば右端に置く  
if ( (色 == 赤) && (数字 == 偶数) ){  
    右端に置く;  
}

// カードが黒の奇数ならば左端に置く  
else if ( (色 == 黒) && (数字 == 奇数) ){  
    左端に置く;  
}

// それ以外は裏向けて真ん中に置く  
else{  
    裏向けて真ん中に置く;  
}

## 演算子の優先順位と結合規則

| 演算子               | 優先順位         | 結合規則 |
|-------------------|--------------|------|
| (, )              | ( )内を先に計算する  |      |
| ++, --            | 次に++,--を計算する |      |
| *, /, %           | 次に四則演算をする    |      |
| +, -              |              |      |
| <, >, <=, >=      | 次に比較演算をする    |      |
| ==, !=            | 次に論理演算する     |      |
| &&,               |              |      |
| =, +=, -=, *=, /= | 最後に代入する      |      |

## if文(多岐条件文)

- 条件が成立／不成立で場合分け
- // 条件〇〇が成り立つならば、処理Aを実行

```
if(〇〇) {
    // 処理A
}
```

文が1つなら{, }は省略可  
(省略しない方がよい)

```
// そうでなく、かつ★★★であれば、処理Bを実行
else if(★★★){
    // 処理B
}
// 処理がなければ省略可能
```

157

## 多岐条件文の文法

```
// 場合1が成立すれば、
// 以下の処理を実行する
if (場合1) {
    // 処理1
}
```

```
// 場合2が成立すれば、
// 以下の処理を実行する
else if (場合2) {
    // 処理2
}
```

```
// 場合3が成立すれば、
// 以下の処理を実行する
else if (場合3) {
    // 処理3
}
```

```
// どの場合も成立しないなら
// 以下の処理を実行する
else {
    // 処理n
}
```

158

## 例

変数scoreに  
点数が入っている  
点数に応じて  
秀, 優, 良, 可, 不可  
を表示するには?

```
if( score >= 90 ) {
    cout << "秀!!" << endl;
} else if ( score >= 80 ) {
    cout << "優!" << endl;
} else if (score >= 70 ) {
    cout << "良" << endl;
} else if (score >= 60 ) {
    cout << "可" << endl;
} else {
    cout << "不可" << endl;
}
```

else ifを使わずに、if文だけを並べて書くとうなる?

159

```
if( score >= 90 ) {
    cout << "秀!!" << endl;
}
if ( (90 > score) && (score >= 80) ) {
    cout << "優!" << endl;
}
if ( (80 > score) && (score >= 70) ) {
    cout << "良" << endl;
}
if ( (70 > score) && (score >= 60) ) {
    cout << "可" << endl;
}
if( 60 > score ) {
    cout << "不可" << endl;
}
```

if文だけでも書けるが  
バグの元なので避ける

160

## 入れ子if文

- if文の中で別のif文を使う  
(書けるならばswitch文で書くこと)
- 入れ子if文を使う時には注意が必要
  - ifとelseの対応があいまいになりやすい
  - elseは直近のif文に対応する
  - { }を省略せずに、対応を明確にする
- 条件判定で、すべての場合が出尽くしていることを確認する

161

## 入れ子if文によるあいまいさの例

- 例1: あいまいな例
- あいまいさを避けるには括弧を使う

```
if (条件1)
    if (条件2) {
        対応
    }
    else {
        対応
    }

if (条件1) {
    if (条件2) {
        対応
    }
    :
}
else {
    対応
}
```

elseはif (条件1)と対応しているように見える(実は違う)

162



## 答え合わせ

全ての答えをメモしましたか？

## 解答

- 【1】 ④  $c = 2 + (4 / 2)$
- 【2】 ②  $b = 2 + (4 \% 4)$  ※4%4は余り0
- 【3】 ③  $a = 2++;$
- 【4】 ①  $a = 3 * (2 - 2)$
- 【5】 ②  $1++$
- 【6】 ③  $1+2$
- 【7】 ③  $1*3$
- 【8】 ①  $1--$
- 【9】 ①  $10 \% 3$ (余りは1)

## 解答

【問10】 #define PI 3.141592  
(セミコロンを付けないように)

【問11】 int value = 123;

## 解答

【問12】

```
void main (){
    int a, b, c;
    a = 3;
    b = 5;
    c = 2;
    b = 0;
    c = a;
    a = b;
    b = c;
}
```

| 変数<br>ステップ | int a; | int b; | int c; |
|------------|--------|--------|--------|
| a = 3;     | 3      | —      | —      |
| b = 5;     | 3      | 5      | —      |
| c = 2;     | 3      | 5      | 2      |
| b = 0;     | 3      | 0      | 2      |
| c = a;     | 3      | 0      | 3      |
| a = b;     | 0      | 0      | 3      |
| b = c;     | 0      | 3      | 3      |

## 解答

【問13】

```
void main (){
    int a=0, b=1, c=2,
        d=3, e=4, f=5;
    a = b;
    b = d;
    d = a;
    c = e;
    f = c;
    e = f;
    a = c;
}
```

| 変数<br>ステップ | int a; | int b; | int c; | int d; | int e; | int f; |
|------------|--------|--------|--------|--------|--------|--------|
| 初期化        | 0      | 1      | 2      | 3      | 4      | 5      |
| a = b;     | 1      | 1      | 2      | 3      | 4      | 5      |
| b = d;     | 1      | 3      | 2      | 3      | 4      | 5      |
| d = a;     | 1      | 3      | 2      | 1      | 4      | 5      |
| c = e;     | 1      | 3      | 4      | 1      | 4      | 5      |
| f = c;     | 1      | 3      | 4      | 1      | 4      | 4      |
| e = f;     | 1      | 3      | 4      | 1      | 4      | 4      |
| a = c;     | 4      | 3      | 4      | 1      | 4      | 4      |
| 答え         | 4      | 3      | 4      | 1      | 4      | 4      |

お疲れさまでした

